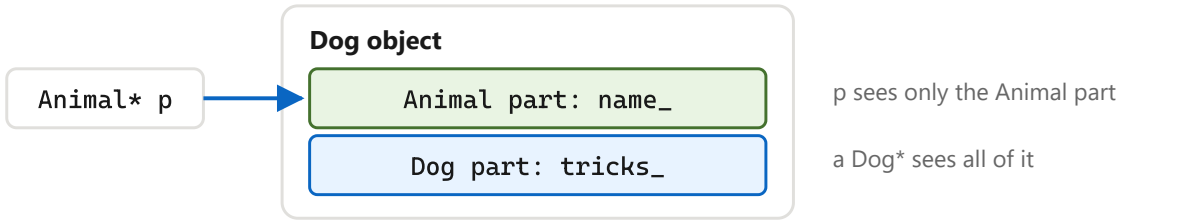
Inheritance, Virtual Functions and the Object Model

Public inheritance as is-a, how a virtual call finds the right function (the vtable and the vptr), abstract classes, composition, and what lies beneath: multiple and virtual inheritance, RTTI and private inheritance. Assumes Copy Control and Operators.

Inheritance

Public inheritance models **is-a**: a `Dog` works anywhere an `Animal` is expected.



A derived object is the base part plus its own members. That is why a Dog can go wherever an Animal is expected.

Inheritance is containment with a promise: it behaves like the base.

```
class Animal {
public:
    explicit Animal(std::string name) : name_(std::move(name)) {}
    const std::string& name() const { return name_; }
protected:
    // visible to Dog, hidden from the outside world
    std::string name_;
};

class Dog : public Animal {
public:
    // call the base constructor
    explicit Dog(std::string name) : Animal(std::move(name)) {}
    void fetch() const { std::cout << name_ << " fetches\n"; }
};
```

> **OBJECT SLICING** `Animal a = Dog("Rex");` chops off the Dog part and its virtual behaviour. Use pointers or references.

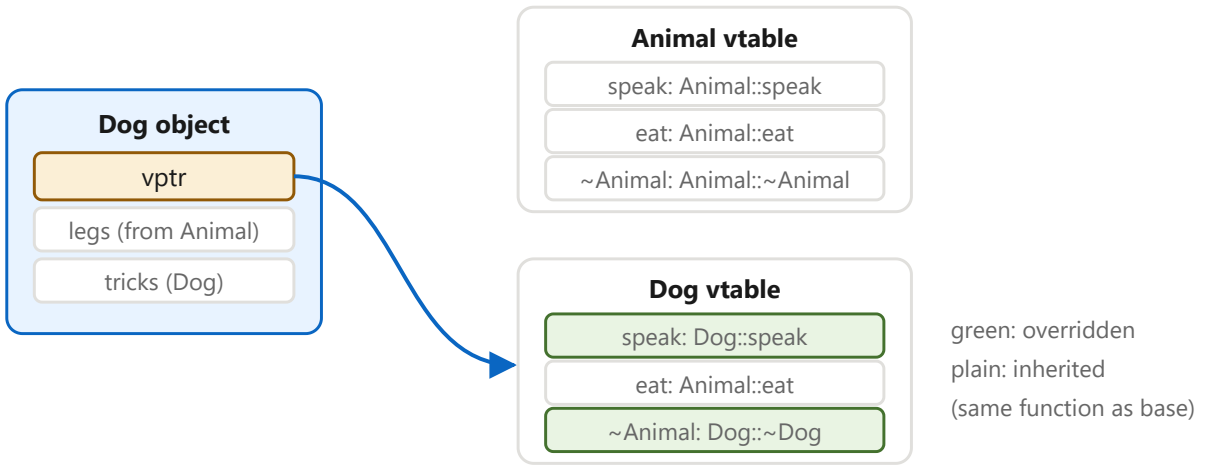
`final` forbids deriving. Prefer **composition** unless it is truly is-a.

Polymorphism and virtual functions

`p->speak()` through a base pointer reaches the *derived* behaviour only if `speak` is `virtual`. Without it the **static type** (the type of the pointer, known at compile time) decides; with it the **dynamic type** (the type of the object, known at run time) decides.

How it works: vtable and vptr

The compiler builds one **vtable** per class that has virtual functions: an array of function pointers, one **slot** per virtual function. A slot holds the *most derived override* of that function. A derived class's vtable starts as a copy of its base's, and every function the derived class overrides replaces the matching slot. Every object of such a class carries one hidden pointer, the **vptr**, normally at the start of the object, set to its class's vtable by the constructor.



One vtable per class, shared by all its objects; one vptr per object. Only the overridden slots differ.

- **Cost.** A class with virtual functions has a vptr in every object: `struct Plain { int legs; }` is 4 bytes, the same struct with one virtual function is 16 on a typical 64-bit platform (8 for the vptr, 4 for `legs`, 4 of padding). The data members come after the vptr.

- **The call.** `p->speak()` compiles to three steps: load the vptr from the object, index the vtable at the slot number that `speak` has in `Animal`, and jump to the address found there. The slot number is fixed at compile time because every derived vtable keeps the base's slot order. That is also why virtual calls usually cannot be inlined.
- **Only virtual functions have a slot.** Non-virtual functions are called directly, chosen by the static type.
- **A virtual destructor takes a slot too.** That is how `delete p` through an `Animal*` runs `Dog::~Dog`.
- **A pure virtual function** (`= 0`) still has a slot, but it holds no useful code; that is why the class cannot be instantiated.

Writing it: `override`, `final` and a virtual destructor

```
struct Shape {
    virtual double area() const = 0;
    // ALWAYS virtual in a polymorphic base
    virtual ~Shape() = default;
};
struct Circle : Shape {
    double r;
    explicit Circle(double r) : r(r) {}
    // `override`: the compiler checks you really override
    double area() const override { return 3.14159 * r * r; }
};
// `final` on a class: nothing can derive from it
struct Ball final : Circle {
    using Circle::Circle;
};
// `final` on a function stops further overriding: double area() const
// final;
```

TWO FAMOUS TRAPS

1. Always write `override`. If you mistype the signature (forget `const`), without it you silently create a *new* function instead of overriding.

2. Virtual calls inside a constructor or destructor do **not** reach the derived class. The constructor of `Base` sets the vptr to `Base`'s vtable, and only *after* it finishes does the constructor of `Derived` set it to `Derived`'s vtable. So while `Base`'s constructor runs, the object's dynamic type is `Base`, and `Base::init()` is called even if `Derived` overrides it. A destructor does the same in reverse.

Overloading, overriding and hiding

TERM	MEANING
Overloading	same name, different parameters, resolved at compile time
Overriding	a derived class replaces a *virtual* base function with the same signature, resolved at runtime
Hiding	a derived function with the same name (but not virtual / different signature) hides *every* base function of that name

```
struct B { void f(int); void g(); };
// hides B::f(int), it does not overload it
struct D : B { void f(double); };

D d;
// calls D::f(double): the int overload of B is not even considered
d.f(1);
d.g();         // fine: D has no g, so B::g is found
d.B::f(1);     // explicit qualification reaches the hidden one
// or add `using B::f;` inside D to bring B's overloads back into D's
// scope
```

Multiple base classes, the diamond problem, `dynamic_cast` and the exact object layout are in the sections further down this lesson.

Abstract classes and interfaces

`= 0` makes a function **pure virtual**; a class with one is **abstract** (cannot be instantiated). Only pure virtuals and no data: an **interface**.

```
struct Logger {                                    // interface
    virtual void log(const std::string& msg) = 0;
    virtual ~Logger() = default;
};
struct ConsoleLogger : Logger {
    void log(const std::string& msg) override { std::cout << msg << '\n'; }
};

// Code depends on the abstraction, not the concrete class:
void run(Logger& out) { out.log("started"); }

// Polymorphic collection: unique_ptr owns the objects, no slicing, no
// manual delete
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(2.0));
for (const auto& s : shapes) std::cout << s->area() << '\n';
```

Composition, relationships and design



Prefer composition; inheritance is the strongest coupling.

"Is it a, or does it have a?" Prefer **composition** unless you need polymorphism.

RELATIONSHIP	MEANING	C++ SHAPE	LIFETIME
Inheritance (is-a)	Dog is an Animal	`class Dog : public Animal`	one object
Composition (owns)	Car has an Engine, which dies with the car	member `Engine e;`	part dies with whole
Aggregation (uses, shared)	Team has Players who exist independently	`Player*` / `std::shared_ptr` member	independent
Association	Doctor treats Patients	reference or pointer parameter	independent

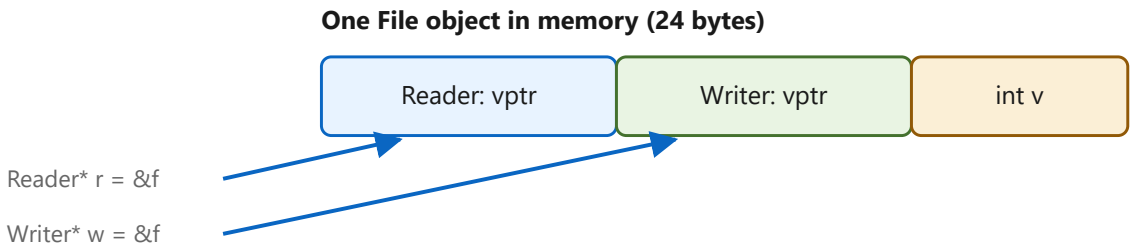
> LISKOV SUBSTITUTION Code using a base must keep working with any derived type. If `Square : Rectangle` breaks on setting the width, inheritance was the wrong model: it must mean "behaves like", not "shares code".

Related notes in your repo: Module 2: C++ OOP, objects relationships, and the learncpp.com recaps for chapters 14, 15, 21, 23, 24 and 25.

Multiple inheritance and object layout

A class may have several bases. The sensible use is **several pure interfaces** (abstract classes with no data), as in Java:

```
struct Reader { virtual int read() = 0;        virtual ~Reader() = default;
};
struct Writer { virtual void write(int) = 0;   virtual ~Writer() = default;
};
// implements both interfaces
struct File : Reader, Writer {
    int v = 0;
    int read() override { return v; }
    void write(int x) override { v = x; }
};
```

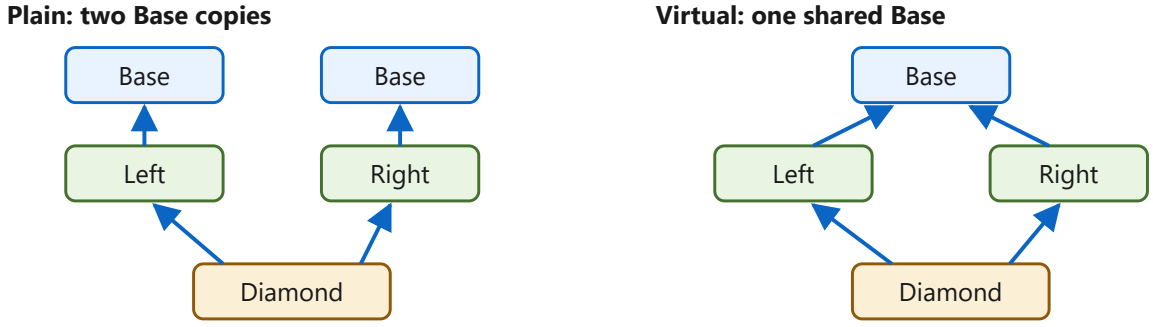


Converting File* to Writer* adjusts the address: the two pointers differ (compiled).

Each polymorphic base brings its own vptr, so the pointer value changes when you convert to the second base.

- The object is the bases laid out one after another, then the derived members. Compiled: `sizeof(File)` is **24** bytes (two vptrs and an `int`).
- Converting `File*` to the **second** base adjusts the pointer value (`static_cast` and implicit conversion do it for you). That is why you must not cast with `reinterpret_cast` or a C cast.
- Both bases are polymorphic, so `delete` through either needs a **virtual destructor**.
- **Name clashes:** if both bases have a member `f`, calling `obj.f()` is ambiguous. Qualify it (`obj.Reader::f()`) or override it in the derived class.
- Multiple inheritance of *implementation* (bases with data) is where trouble starts: the diamond.

The diamond problem and virtual inheritance



Plain: d.id is ambiguous (two ids). Virtual: one id.

Virtual inheritance makes the shared base appear once.

```
struct Base  { int id; };
// "virtual" goes on the MIDDLE classes
struct Left  : virtual Base {};
struct Right : virtual Base {};
// one Base inside; d.id is not ambiguous
struct Diamond : Left, Right {};
```

	PLAIN INHERITANCE	VIRTUAL INHERITANCE
Base subobjects in `Diamond`	two (compiled `sizeof` 8: two ids)	one (compiled `sizeof` 24: one id plus the machinery)
`d.id`	ambiguous: write `d.Left::id`	fine
Constructor order (compiled)	`Base Left Base Right Diamond`: Base built twice	`VBase VLeft VRight VDiamond`: shared base once, **first**
Who initializes the base?	each middle class	the **most derived** class, directly
Cost	none	extra pointer(s) per object, indirect access to the base, slower casts

- The standard library's `std::iostream` is the classic use: `istream` and `ostream` inherit `ios` virtually, so `iostream` has one stream state.
- **Advice:** avoid diamonds. Use interfaces without data, or composition. When you need virtual inheritance, put the data in one place and let the most derived class initialize the base.

KEY POINT "What is the diamond problem and how do you solve it?" Two paths to one base duplicate its members and make access ambiguous. `virtual` inheritance shares one copy; the most derived class constructs it first. It costs memory and indirection, so prefer interfaces and composition.

RTTI: `typeid` and `dynamic_cast`

RTTI (run-time type information) lets you ask for the *dynamic* type of an object behind a base pointer. It works only for **polymorphic** classes (at least one virtual function), because the type information lives next to the vtable.

```
Animal* a = new Dog;
// true: the dynamic type (compiled)
typeid(*a) == typeid(Dog);
typeid(*a) == typeid(Cat);                    // false
// an implementation-defined name, for debugging only
typeid(*a).name();

// set only if *a really is a Dog (or derived from Dog)
if (Dog* d = dynamic_cast<Dog*>(a)) { }
// nullptr: a Dog is not a Cat
Cat* c = dynamic_cast<Cat*>(a);
// a reference cannot be null: throws std::bad_cast
Cat& r = dynamic_cast<Cat&>(*a);

// a CROSS-cast: sideways between two bases of the same object
Writer* w = dynamic_cast<Writer*>(readerPtr);
```

QUESTION	ANSWER
What if the class has no virtual function?	`typeid` gives the **static** type; `dynamic_cast` down does not compile
Cost?	`dynamic_cast` walks the class hierarchy at run time: slower than a virtual call, avoid it on hot paths
`static_cast` down instead?	no check: undefined behaviour if you are wrong
`-fno-rtti`	compilers can turn RTTI off to save space; `typeid` and `dynamic_cast` then disappear (some embedded and low-latency code does this)

> **A SMELL** Chains of `if (dynamic_cast<A*> ... ) else if (dynamic_cast<B*> ... )` usually mean a missing virtual function or a Visitor.

Private inheritance versus composition

KIND	MEANING	CONVERTS TO BASE OUTSIDE?
`public` inheritance	**is-a**: usable wherever a `Base` is	yes
`private` inheritance	**implemented-in-terms-of**: an implementation detail	no (compiled: `Car*` to `Engine*` is not convertible)
`protected` inheritance	like private, but derived classes of the derived class see the base	no

```
struct Car : private Engine {
    void drive() { start(); }                 // uses the base internally
    // re-export one member if you want to
    using Engine::start;
};
// Car c; Engine* e = &c; // ERROR: the base is private
```

- **Prefer composition** (a member `Engine engine_;`): it is simpler and does not expose the base's virtual functions to accidental overriding. Use private inheritance only to override a virtual function of the base, access a `protected` member, or get the **empty base optimisation** (an empty base takes no space; a member takes at least 1 byte).
- Never inherit publicly just to reuse code: if it is not *is-a*, it breaks the Liskov substitution principle.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Inheritance	C++ Primer	15.1 to 15.2 OOP overview, defining base and derived classes	621 to 631
	C++ Primer	15.5 Access control and inheritance	640 to 645
	Programming: Principles and Practice	12.3 Base and derived classes (object layout, the vptr and vtbl, overriding)	380 to 387
	Programming: Principles and Practice	15.5 Destructors, including 15.5.2 Virtual destructors	460 to 463
	C++ Move Semantics	4.4 Move semantics in class hierarchies	93 to 96
Polymorphism and virtual functions	C++ Primer	15.3 Virtual functions	632 to 636
	C++ Primer	15.6 Class scope under inheritance	646 to 650
	C++ Primer	15.8 Containers and inheritance	659 to 662
	Effective Modern C++	Item 12, Declare overriding functions override	97 to 104
	C++ Move Semantics	4.4 Move semantics in class hierarchies	93 to 96
Abstract classes and interfaces	C++ Primer	15.4 Abstract base classes	637 to 639